

Personal Knowledge Archive

Architecture and Implementation Reference

A local-first, fully offline system for ingesting, indexing, and clustering a multi-source research library using open-weight models.

Sources	Firefox bookmarks, Zotero library, book archive (PDF/EPUB)
Interface	Local web application
Infrastructure	Fully local; GPU-accelerated; no cloud dependencies
Source policy	Read-only on all source systems
Tag policy	Overlay database; existing filings preserved

Contents

1	System overview	2
2	Source connectors	2
2.1	Firefox bookmarks	2
2.2	Zotero	3
2.3	Book archive	3
3	Ingestion pipeline	3
3.1	Text cleaning and chunking	3
3.2	Embedding model selection	4
4	Storage layer	5
4.1	Vector store	5
4.2	Metadata and overlay database	5
4.2.1	Core tables (read-derived, not editable)	5
4.2.2	Overlay tables (system-derived and user-editable)	5
5	Clustering and organisation engine	6
5.1	Clustering pipeline	6
5.1.1	Dimensionality reduction	6
5.1.2	Geometric clustering	6
5.1.3	Cluster labelling	6
5.2	Clustering algorithm options	7
5.3	Diagnostics before acceptance	7
5.4	Cluster lifecycle management	7
5.5	Interest timelines	7
6	Web interface	8
6.0.1	Interface views	8
7	Dependency summary	9
8	Deployment notes	9

System overview

The system aggregates three heterogeneous document sources into a single queryable knowledge base. It does not modify any source system. All derived data — embeddings, inferred tags, cluster assignments, reading lists, and cached content — lives in a dedicated local database layer.

The architecture has five logical layers:

1. **Source connectors** — read-only adapters for Firefox, Zotero SQLite, and the file system.
2. **Ingestion pipelines** — text extraction, cleaning, chunking, and embedding.
3. **Storage layer** — a vector store for semantic search and a relational store for metadata, tags, and provenance.
4. **Clustering and organisation engine** — offline topic inference, cluster lifecycle management, and timeline construction.
5. **Web interface** — search, browse, tag editor, cluster explorer, and reading list builder.

Figure 1 shows the high-level data flow.

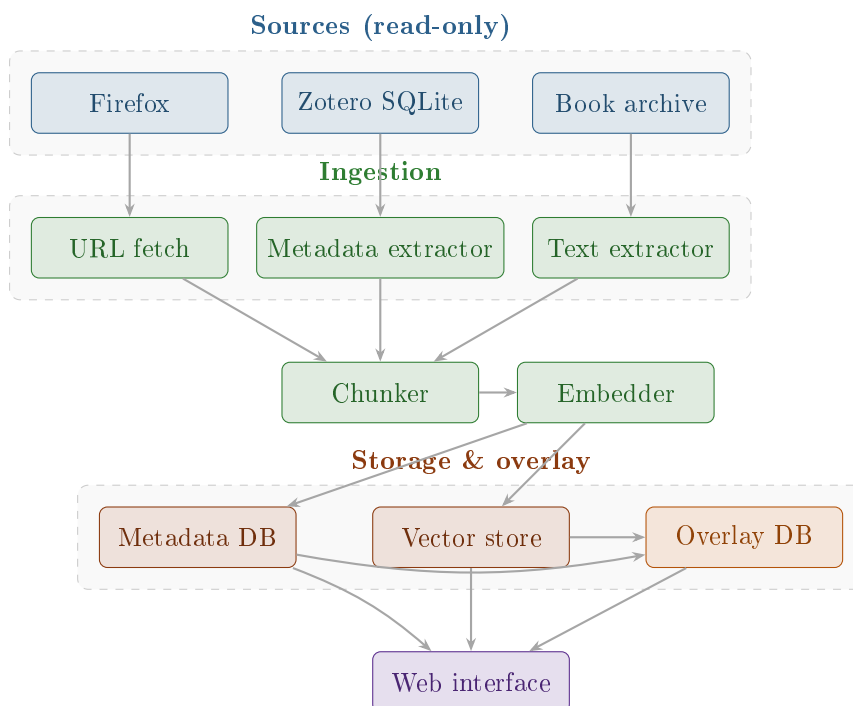


Figure 1: High-level data flow. All arrows represent read or write operations within the local system. No network calls leave the machine except during URL fetching (bookmark ingestion only).

Source connectors

Firefox bookmarks

Firefox stores bookmarks in a SQLite database at `~/.mozilla/firefox/<profile>/places.sqlite`.

The connector queries this file directly (read-only, with a file copy to avoid lock contention) and retrieves: URL, title, date added, last modified, bookmark folder path, and any tags assigned in

Firefox. It does not require a browser extension.

Content fetching is a separate, asynchronous step. For each URL the system attempts an HTTP GET with a reasonable timeout. Outcomes:

- **Success** — HTML is extracted (main content via `trafilatura` or `readability-lxml`), stored in the cache, and marked `fetch`d.
- **Flagged** — URLs returning 4xx/5xx, requiring authentication, or timing out are stored with status `unfetchable` and surfaced in the interface for manual review. The bookmark record is retained and searchable by metadata.
- **Skipped** — Non-HTML targets (e.g. PDF direct links, torrents) are handled separately or flagged.

Given thousands of bookmarks, fetching should be rate-limited and run as a background job, not blocking the UI.

Zotero

Zotero's local database is at `~/Zotero/zotero.sqlite` (default path, configurable). The connector queries it read-only using a temporary copy.

Key tables: `items`, `itemData`, `fields`, `collections`, `collectionItems`, `tags`, `itemTags`, `itemAttachments`.

Extracted per item: title, authors, abstract, year, DOI, item type, Zotero collections, Zotero tags, and the path to any attached PDF. The connector preserves the original Zotero tag and collection strings verbatim in the metadata database; these are never modified.

Book archive

The connector walks the archive directory recursively and indexes all `.pdf` and `.epub` files. For each file:

- **PDF** — metadata extracted via `pypdf`; chapter structure inferred from bookmarks/outline (`/Outlines` in the PDF object tree) when present; first page of each chapter extracted as text.
- **EPUB** — metadata from `OPF` manifest; chapter list from `NCX/nav` document; first n characters of each chapter extracted.

Full-text extraction is deferred to a second pass (triggerable per document or in batch) to keep initial indexing fast.

Ingestion pipeline

Text cleaning and chunking

All extracted text passes through a shared cleaning step (whitespace normalisation, de-hyphenation for PDFs, encoding fixes) before chunking.

Chunking strategy options:

Strategy	Advantages	Drawbacks	Fit
Fixed-size (e.g. 512 tokens, 64 overlap)	Simple, fast, predictable index size	Cuts across sentences and arguments	Baseline
Sentence-window (group k sentences)	Preserves semantic units better	Slightly more complex; variable chunk size	Good default
Recursive character split (LangChain-style)	Adapts to paragraph structure	Heuristic; no semantic awareness	Acceptable
Semantic chunking (embed + split on cosine drop)	Splits at genuine topic boundaries	Slower; requires embedding during chunking	Best quality, higher cost

Table 1: Chunking strategy comparison. Sentence-window chunking is a reasonable default; semantic chunking can be applied selectively to longer books.

Embedding model selection

All models run locally via Ollama. Dimensions and speed figures are approximate and GPU-dependent.

Model	Dims	Relative speed	Notes
nomic-embed-text	768	Fast	Strong general performance; well-tested with Ollama; good default
mxbai-embed-large	1024	Medium	Higher quality on retrieval benchmarks; larger index footprint
bge-large-en-v1.5	1024	Medium	Strong English performance; less multilingual than bge-m3
bge-m3	1024	Slower	Best for mixed-language archives; unnecessary overhead for English-primary
all-minilm-16-v2	384	Very fast	Lightweight; acceptable for large archives where speed is critical

Table 2: Embedding model options available via Ollama. For a primarily English archive, `nomic-embed-text` or `mxbai-embed-large` cover the practical range from fast to high-quality.

Storage layer

Vector store

Option	Advantages	Drawbacks	Fit
Chroma	Embedded (no server), Python-native, simple API, persistent on disk	Less scalable beyond 1M vectors	Strong fit
Qdrant	High performance, rich filtering, REST + gRPC API, Docker deployment	Requires running a server process	Good for larger archives
LanceDB	Columnar storage, embedded, very fast scans	Younger project, smaller ecosystem	Viable alternative
FAISS	Extremely fast, battle-tested	No persistence out of the box; no metadata filtering	Low-level only

Table 3: Vector store options. At low thousands of documents with moderate chunk counts, Chroma is sufficient and requires no server management. Qdrant becomes preferable if the archive grows significantly or if complex metadata filtering is needed at query time.

Metadata and overlay database

A single SQLite database (`archive.db`) handles all structured data. The schema is divided into two namespaces: *core* tables (populated from source connectors, never edited by the user) and *overlay* tables (the system’s own derived data).

Core tables (read-derived, not editable)

- **documents** — one row per source item; fields include `source` (`firefox/zotero/archive`), `source_id`, `title`, `url_or_path`, `date_added`, `fetch_status`.
- **source_tags** — tags as they exist in the source system (`document_id`, `tag_string`, `source`).
- **source_collections** — collection/folder membership from Zotero and Firefox folder hierarchy.
- **chunks** — one row per text chunk; foreign key to **documents**; stores chunk text, token count, and vector store ID.
- **fetch_log** — per-URL fetch attempts with timestamp, HTTP status, and error message.

Overlay tables (system-derived and user-editable)

- **overlay_tags** — (`document_id`, `tag`, `origin`: `inferred/manual`, `confidence`, `created_at`).
- **clusters** — (`cluster_id`, `label`, `description`, `created_at`, `run_id`).
- **cluster_assignments** — (`document_id`, `cluster_id`, `run_id`, `score`, `assigned_at`). Assignments from past runs are retained; only the latest run is “active”.
- **cluster_runs** — metadata per clustering run: timestamp, algorithm, parameters, accepted (boolean), notes.
- **reading_lists** — (`list_id`, `name`, `description`, `created_at`).

- `reading_list_items` — (`list_id`, `document_id`, `position`, `note`).

This structure ensures that rolling back to a previous clustering state requires only changing which `run_id` is marked active, with full history retained.

Clustering and organisation engine

Clustering pipeline

Topic clustering operates on document-level embeddings (averaged or weighted from chunk embeddings) rather than chunk embeddings directly, to produce document-granularity cluster assignments.

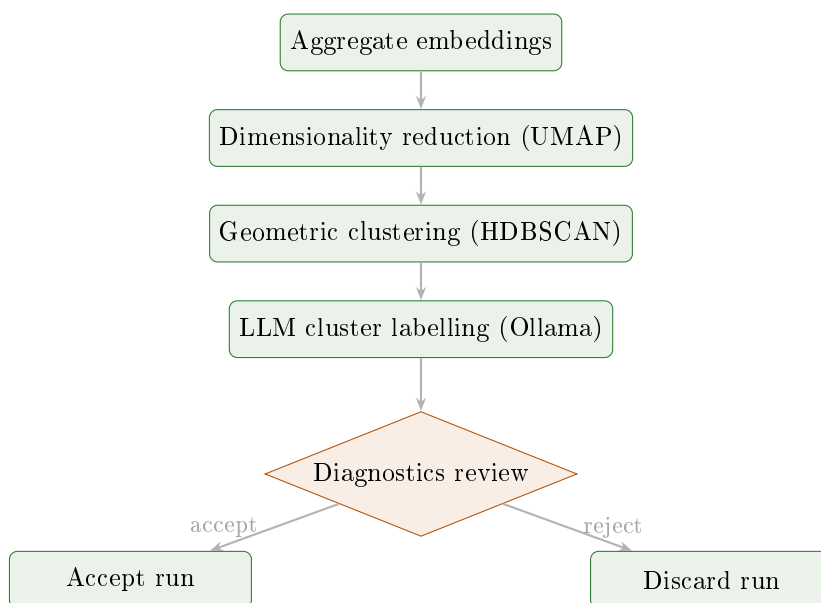


Figure 2: Clustering pipeline. The run is stored regardless of acceptance; acceptance only controls which run is treated as the active assignment.

Dimensionality reduction

UMAP (`umap-learn`) reduces high-dimensional embeddings (768–1024 dims) to 2–5 dimensions before clustering. This improves HDBSCAN performance and also produces a 2D projection usable for visualisation in the interface. Key parameters: `n_neighbors` (local vs. global structure), `min_dist` (cluster tightness in projection). Both should be exposed in the interface for the user to adjust before triggering a run.

Geometric clustering

HDBSCAN (`hdbscan` or `scikit-learn` ≥ 1.3) is preferred over k-means for this use case because it does not require specifying the number of clusters in advance, handles variable cluster density, and explicitly marks outlier documents (noise points) rather than forcing assignment. Key parameter: `min_cluster_size`.

Cluster labelling

After geometric clustering, a local Ollama model (e.g. `llama3`, `mistral`, or `phi3`) receives a sample of titles and abstracts from each cluster and is prompted to produce a short label and a

one-sentence description. This step is optional and can be re-run independently of the geometric step.

Clustering algorithm options

Algorithm	Advantages	Drawbacks	Fit
HDBSCAN	No fixed k ; handles noise; density-aware	Sensitive to <code>min_cluster_size</code> ; can over-fragment	Strong fit
k-means	Fast, deterministic	Requires fixed k ; assumes spherical clusters	Weak fit
Agglomerative	Produces dendrogram; flexible linkage	Quadratic memory; no noise handling	Partial fit
BERTopic	End-to-end pipeline; good defaults	Opinionated; harder to customise	Viable
Gaussian Mixture	Soft assignments (probabilities)	Requires fixed k ; less interpretable	Optional

Table 4: Clustering algorithm comparison. HDBSCAN is the recommended default. BERTopic is a packaged alternative that bundles UMAP + HDBSCAN + TF-IDF labelling and may reduce implementation effort at the cost of flexibility.

Diagnostics before acceptance

Before a clustering run can be accepted, the interface presents:

- Number of clusters found; number of noise/unassigned documents.
- Per-cluster: size, top terms (TF-IDF), LLM label, sample titles.
- 2D UMAP scatter plot, coloured by cluster assignment.
- Comparison with the previous active run: documents that changed cluster, clusters that split or merged, new documents added since last run.
- Distribution of cluster sizes (histogram).

The user can accept, reject, or partially accept (promote individual cluster assignments while keeping previous assignments for others — an advanced option).

Cluster lifecycle management

New documents added after an accepted run are assigned to the nearest existing cluster using cosine similarity to cluster centroids. This does not trigger a full re-run.

Periodic re-runs can be triggered manually. The system computes a *drift score* per cluster (mean distance of recent additions from the centroid) and flags clusters that may warrant splitting. It also computes pairwise cluster similarity and flags pairs that may warrant merging. These are displayed as suggestions, not automatic actions.

Interest timelines

Because `date_added` is recorded for all sources (bookmark creation date, Zotero import date, file modification date as fallback), it is possible to plot cluster membership over time. The interface

can display a stacked timeline showing how many items in each cluster were added per month or year. This surfaces patterns such as periods of concentrated reading in a topic, or topics that recur at intervals.

Web interface

The interface is a local web application. The recommended stack is a lightweight Python backend serving a JavaScript frontend.

Backend option	Advantages	Drawbacks	Fit
FastAPI	Async, fast, auto-docs, Python-native	Slightly more setup than Flask	Strong fit
Flask	Simple, minimal	Sync by default; less modern	Acceptable

Frontend option	Advantages	Drawbacks	Fit
Svelte / SvelteKit	Lightweight, reactive, minimal boilerplate	Smaller ecosystem than React	Strong fit
React + Vite	Large ecosystem, component libraries	Heavier; more configuration	Good fit
HTMX + Jinja2	Minimal JS; Python templates	Less dynamic; harder for complex UI	Partial fit

Table 5: Backend and frontend options. FastAPI + Svelte is a reasonable combination for a local tool with moderate UI complexity.

Interface views

- **Search** — full-text and semantic search across all sources; filters by source, tag (source or overlay), cluster, date range, fetch status.
- **Document detail** — metadata, source tags, overlay tags (editable), cluster assignment, reading list membership, cached content preview.
- **Cluster explorer** — list of clusters with labels and sizes; scatter plot (UMAP 2D); per-cluster document list; interest timeline.
- **Cluster run manager** — run history table; diagnostics panel; accept/reject controls; drift and merge/split suggestions.
- **Tag browser** — unified view of source tags and overlay tags; tag frequency; documents per tag.
- **Reading lists** — create, rename, add/remove documents; reorder; export.
- **Ingestion status** — per-source sync state; unfetchable bookmark list; background job progress.

Dependency summary

Component	Library / tool	Notes
Ollama	ollama (local server)	Embedding and LLM inference
Vector store	chromadb or qdrant-client	See Section 4.1
Metadata DB	sqlite3 (stdlib)	No external server
PDF extraction	pypdf, pdfplumber	Fallback: pdftotext
EPUB extraction	ebooklib	
HTML extraction	trafilatura	Primary; fallback: readability-lxml
Embeddings	nomic-embed-text via Ollama	Or mxbai-embed-large
Dimensionality reduction	umap-learn	
Clustering	hdbscan / scikit-learn	
LLM labelling	ollama Python client	llama3 or mistral
Backend	fastapi, uvicorn	
Frontend	Svelte / SvelteKit	Or React + Vite
HTTP fetching	httpx	Async; respects rate limits
Scheduling	APScheduler or rq	Background jobs

Table 6: Recommended dependency stack. All components are open source. No component requires a network connection after initial model download.

Deployment notes

The system runs as a set of local processes:

1. **Ollama server** — started separately; handles all embedding and LLM requests on the local GPU.
2. **Vector store server** (if Qdrant) — Docker container or native binary. Not needed if Chroma is used in embedded mode.
3. **FastAPI backend** — serves the API and static frontend files.
4. **Worker process** — handles background ingestion jobs (fetching, embedding) asynchronously to avoid blocking the UI.

A simple shell script or `docker-compose.yml` can start processes 2–4 together. Process 1 (Ollama) is typically already running as a system service.

Data directories that should be configurable: Zotero database path, Firefox profile path, book archive root, cache and database output directory.